SIMATS ENGINEERING

Department of Computer Science and Engineering

Compiler Design – Unit I: Introduction to Compilers and Lexical Analysis ASSIGNMENT 1 – CO1

Development of a Domain-Specific Compiler Front-End using LEX for Federated Cyber Threat Detection in Security Operations Centers

CO1: Compiler fundamentals and language processors; phases of a compiler and grouping of phases; compiler construction tools; role of the lexical analyzer; specification and recognition of tokens; and the language for specifying lexical analyzers (LEX).

Student Name	E.RAGUL	**Register Number**	192571032
Department / Section	CSE	**Academic Year / Semester**	AUG-DEC 2026
Faculty Name	Dr.N.Deepa	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	16/11/2026	**Date of Evaluation**	

https://github.com/192571032simats-bit/CSA1409---COMPILER-DESIGN/tree/main/Assignment-1-CO1-ThreatRule-Compiler



TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Introduction	3
2	Problem Statement	3
3	Assignment Objectives	3
4	Scope of the Project	4
5	Tools and Technologies	4
6	Task 1 – Language Processors	5
7	Task 2 – Compiler Phases	7
8	Task 3 – Grouping of Compiler Phases	10
9	Task 4 – Compiler Construction Tools	11
10	Task 5 – Lexical Analyzer	13
11	Task 6 – Token Specification	15
12	Task 7 – Token Recognition	18
13	Task 8 – LEX Specification	20
14	Task 9 – Implementation and Testing	22
15	Task 10 – Performance Evaluation	24

16	**Task 11 – Final Design and Recommendation**	**28**
17	**Task 12 – Day-4 Live-Feed Challenge (12A–12D)**	**30**
18	**CO1 Syllabus Mapping**	**—**
19	**Implementation Evidence Chain**	**36**
20	**Mandatory Submission Evidence Check**	**37**
21	**Submission Checklist**	**38**
22	**Evaluation Rubric – Assignment 1**	**40**
23	**Appendices and References**	**41**
24	**Blank Mark Sheet**	**43**

Project and implementation boundary

ThreatRule is a small, case-insensitive rule-description language for defensive SOC detection. Its sample program is:

```
RULE ssh_attack IF src_ip == 10.0.0.5 AND port == 22 AND attempts > 5 THEN ALERT "SSH brute force"
;
```

The actual implementation is the Flex scanner (threatrule.l); Flex generates `LEX_Scanner/lex.yy.c`,

1. INTRODUCTION

1) A compiler is a system software component that translates source programs into an equivalent target representation. Compiler construction involves several stages, including lexical analysis, syntax analysis, semantic analysis, intermediate-code generation, optimization, and target-code generation.

2) In this assignment, a domain-specific language named ThreatRule is considered for describing cybersecurity detection rules in Security Operations Centers (SOCs). Multiple branch SOCs generate and exchange detection rules with a central SOC. Before these rules are processed or deployed, their lexical structure must be validated to ensure that the tokens are correctly recognized.

3) This project focuses on developing a lexical analyzer using LEX/Flex. It identifies keywords, identifiers, IP addresses, port numbers, integers, strings, relational operators, and other supported token types. It also demonstrates lexical error detection, recovery, and processing of large rule files.

4) The implementation provides a foundation for a larger ThreatRule compiler front end while explaining the remaining compiler phases as architectural extensions.

2. PROBLEM STATEMENT

1) A federated Security Operations Center receives threat detection rules from multiple branch organizations. These rules may contain incorrectly formatted identifiers, invalid IP addresses, unsupported characters, malformed strings, and other lexical errors.

2) Without a consistent lexical validation mechanism, invalid rule descriptions may interrupt processing and create inconsistencies between different SOC environments.

3) Therefore, a dedicated lexical analyzer is required to recognize valid ThreatRule tokens, detect invalid lexemes, report errors with line numbers, and continue processing wherever possible.

4) The proposed system must support increasing rule volumes, including 100, 1,000, 10,000, and 100,000 rules, while accommodating additional threat-feed information such as file hashes, CVE identifiers, and URLs

3. ASSIGNMENT OBJECTIVES

1. Understand compiler fundamentals and different language processors.
2. Explain the phases and grouping of a compiler.
3. Identify suitable compiler construction tools for ThreatRule.
4. Design regular expressions for recognizing ThreatRule tokens.
5. Develop a lexical analyzer using LEX/Flex.

6. Implement identifier management and lexical error handling.
7. Test valid, invalid, and boundary-condition inputs.
8. Evaluate scanner performance using different rule-file sizes.
9. Extend token recognition to support hashes, CVE identifiers, and URLs.
10. Recommend a suitable lexical analyzer design based on implementation evidence.

4. SCOPE OF THE PROJECT

1) The implementation concentrates on the lexical-analysis stage of a ThreatRule compiler. It includes regular-expression-based token recognition, keyword identification, identifier collection, IPv4 and port validation, string recognition, error reporting, and scanner execution.
2) The project also explains the proposed syntax-analysis, semantic-analysis, intermediate-codegeneration, optimization, and target-code-generation stages.
3) These additional stages are included to demonstrate the complete compiler architecture. They are not claimed as implemented modules in the current scanner.

5. TOOLS AND TECHNOLOGIES

Component	**Technology**	**Purpose**
Lexical analyzer generator	Flex	Generate scanner source code
Programming language	C	Scanner implementation
Compiler	GCC	Compile generated scanner
Source specification	LEX	Define token patterns and actions
Test automation	Project scripts	Generate and execute test cases
Performance measurement	Python time.perf_counter	Measure execution duration
Visualization	SVG charts	Present measured performance

Version control	Git and GitHub	Maintain project source and evidence

TASK 1 — LANGUAGE PROCESSORS

Processor	Input → output	Typical use	Suitability to central SOC
Compiler	Whole source → translated target program	Ahead-of-time translation and diagnostics	**Selected:** validate rule packs before controlled deployment; reusable native engine and batch diagnostics
Interpreter	Source plus input → actions at run time	Interactive scripts	Useful for experimentation, but runtime parsing adds latency and can make deployment behavior dependent on interpreter/runtime availability
Assembler	Assembly language → machine/object code	Low-level instruction translation	Necessary only if a compiler emits assembly; does not translate ThreatRule semantics directly
Hybrid compiler	Source → intermediate/bytecode, then VM/JIT	Portable managed runtimes	Plausible for heterogeneous SOC nodes, at cost of VM and runtime management
Preprocessor	Source directives → expanded source	Includes, macros, conditional compilation	Optional configuration stage; not the core ThreatRule translator
Linker	Object files/libraries → executable	Resolve symbols and combine modules	Used when linking the generated SOC engine, not for rule syntax
Loader	Executable → process image	Load and start a program	Operating-system/runtime service, not a source translator

compiler is chosen because rules can be checked and transformed before deployment, errors can be reported together, and a stable target can be rolled out to federated SOC nodes. This assumes the SOC can tolerate a compile/deploy boundary; a live authoring console may still provide an interpreter-like preview.

1.1 ROLE OF SUPPORTING LANGUAGE PROCESSORS

The supporting language processors perform different responsibilities in the compilation and execution environment.

1) **Preprocessor:** Processes source directives, expands macros, and handles included files before compilation. It can be used if ThreatRule later supports reusable rule definitions.

2) **Compiler:** Translates the source representation into an intermediate or target representation. It is the selected approach for validating and preparing ThreatRule rule sets.
3) **Assembler:** Converts assembly-language instructions into machine code or object code when the compiler uses an assembly-based target.

1.2 JUSTIFICATION FOR THE SELECTED PROCESSOR

A compiler is selected for the central SOC because it allows rule files to be checked before deployment. Errors can be identified during the preparation stage, and a validated representation can be distributed to different branch SOC environments.

A compiler-based approach also supports reusable rule processing, centralized validation, and future integration with different target security engines.

An interpreter-like preview may still be useful during rule authoring, but compilation is preferred for controlled deployment.

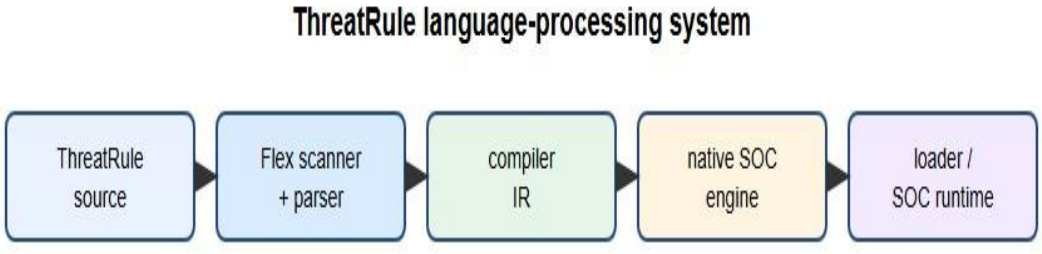
1.3 LANGUAGE PROCESSING SYSTEM

The proposed processing system follows these stages:

1. ThreatRule source files are collected from branch SOC environments.
2. The lexical analyzer recognizes tokens and reports lexical errors.
3. A future parser checks the grammatical structure.
4. A future semantic analyzer validates identifiers, types, and rule constraints.
5. Intermediate representation is generated for validated rules.
6. Optimization improves the intermediate representation where applicable.
7. Target instructions are generated for the selected SOC engine.
8. The resulting engine processes security events during runtime.

Figure 1. Language processing system for the ThreatRule compiler.

The diagram shows source,translator, engine, and runtime; it is an architecture, not evidence that all boxes are implemented.



TASK 2 — COMPILER PHASES

LEXICAL ANALYSIS — ACTUAL GENERATED-SCANNER OUTPUT

The scanner run on the sample recognizes these 18 tokens:

```
RULE RULE
IDENTIFIER ssh_attack
IF IF
IDENTIFIER src_ip
EQ ==
IPV4 10.0.0.5
AND AND
IDENTIFIER port
EQ ==
PORT 22
AND AND
IDENTIFIER attempts
GT >
INTEGER 5
THEN THEN
ALERT ALERT
STRING "SSH brute force"
SEMICOLON ;
```

SYNTAX ANALYSIS — ILLUSTRATIVE TREE

```
Rule(ssh_attack,
  AND(
    AND(Equal(src_ip, IPv4(10.0.0.5)), Equal(port, Integer(22))),
    Greater(attempts, Integer(5))),
  Alert("SSH brute force"))
```

The actual project stops before this phase; no parser output is generated.

SEMANTIC ANALYSIS

Implemented scanner-level checks: every IPv4 octet is 0–255 and a value immediately following the `port` field and relational operator is 0–65535. Those are lexical/attribute checks, not a complete semantic pass. Field declarations, type compatibility, duplicate rule names, and action policy are not CHECKED.

INTERMEDIATE CODE GENERATION AND OPTIMIZATION — ILLUSTRATIVE

Three-address-code sketch:

```
t1 = src_ip == 10.0.0.5
t2 = port == 22
t3 = t1 AND t2
t4 = attempts > 5
t5 = t3 AND t4
if t5 goto alert_ssh_attack
```

Illustrative optimization: common-subexpression elimination would reuse a previously computed identical predicate when one exists. No optimizer runs in this implementation, and this one-rule input offers no demonstrated cross-expression reuse.

Target-code generation — illustrative

```
LOAD_FIELD r1, src_ip
CMP_IPV4  r1, 10.0.0.5
JNE       next_rule
LOAD_FIELD r2, port CMP_U16
r2, 22
JNE       next_rule
LOAD_FIELD r3, attempts
CMP_GT    r3, 5
JNE       next_rule
EMIT_ALERT "SSH brute force"
```

These are abstract SOC-engine instructions, not machine code emitted by this project. A symbol table stores identifier attributes for later phases; this scanner currently stores distinct identifier spellings. An error handler attaches line numbers, records diagnostics, skips the invalid lexeme, and continues scanning where possible.

2.1 OPTIMIZED INTERMEDIATE REPRESENTATION

The intermediate representation can be simplified by separating individual condition evaluations from their combined logical expression. Illustrative optimized representation: t1 = src_ip == 10.0.0.5 t2 = port == 22 t3 = attempts > 5

t4 = t1 AND t2 t5 = t4 AND t3

IF t5 GOTO alert_ssh_attack

The compiler can apply optimization techniques such as constant propagation, dead-code elimination, and common-subexpression elimination when appropriate.

The above representation illustrates a possible optimization stage. No optimization pass is executed by the current lexical analyzer.

2.2 SEMANTIC ANALYSIS REQUIREMENTS

A complete semantic analyzer for ThreatRule would perform the following checks:

1. Verify that referenced fields such as src_ip, port, and attempts are declared.
2. Check whether comparison operators are compatible with operand types.
3. Validate that port values remain within the permitted range.
4. Identify duplicate rule names within the same rule set.
5. Verify that ALERT and BLOCK actions follow the language specification.
6. Check whether logical expressions contain compatible conditions.

The current scanner performs IPv4 octet and context-dependent port-range checks. It does not implement the complete semantic validation described above.

2.3 ROLE OF THE SYMBOL TABLE

The symbol table maintains information about identifiers encountered during compilation. It can store identifier names, data types, scope information, and other attributes required by later compiler phases.

In the current implementation, the scanner stores distinct identifier spellings. Type information, declaration checking, and scope management are future extensions.

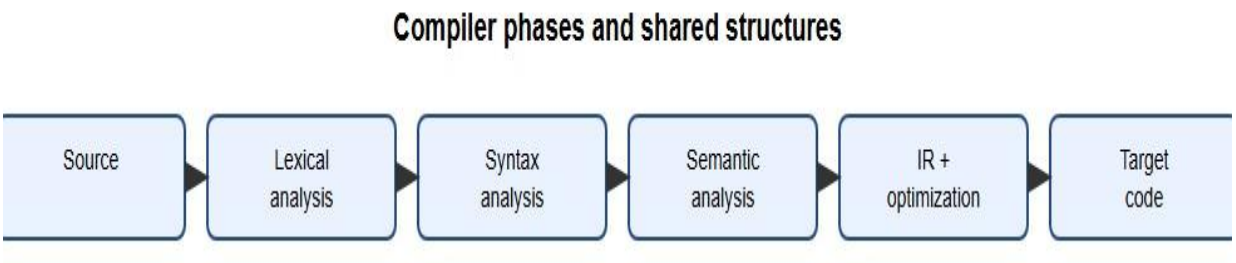
2.4 ROLE OF THE ERROR HANDLER

The error handler identifies invalid input, generates diagnostic messages, and associates errors with source line numbers.

For example, an invalid IPv4 address or unsupported character should produce an appropriate diagnostic. Where recovery is possible, the scanner should continue processing subsequent valid input.

This separation improves the reliability and maintainability of the proposed compiler architecture.

Figure 2:compiler phases.



TASK 3 — GROUPING OF PHASES

The front end comprises scanning, parsing, and semantic checks and is tied to ThreatRule syntax and meaning. The back end consumes an intermediate representation, performs target-oriented optimization, and emits instructions.

An IR boundary allows the same validated rule semantics to target different SOC engines, while branch SOCs can share the language front end.

A *pass* is one complete traversal/analysis of a representation. A multi-pass design is easier to diagnose and extend; a one-pass design may reduce work for simple rules but couples phases and limits optimization.

A staged multi-pass design is the recommended architecture; only the scanning pass is implemented.

3.1 ADVANTAGES OF A COMMON FRONT END

A common front end provides a consistent method of interpreting ThreatRule source files from different branch SOC environments.

Although branch offices may use different security monitoring tools, they can share rules through a common language specification. The central SOC can validate the same token definitions and grammatical conventions before converting validated rules into target-specific representations.

This approach reduces inconsistencies in rule interpretation and makes the system easier to maintain.

3.2 SINGLE-PASS AND MULTI-PASS COMPARISON

Feature	**Single-pass compiler**	**Multi-pass compiler**
Processing	Performs several operations in one traversal	Uses multiple processing stages
Complexity	Can become tightly coupled	Separates responsibilities
Optimization	More limited	Supports additional optimization
Error diagnosis	Less separation between phases	Easier phase-specific diagnosis
Extensibility	More difficult for complex languages	Easier to extend

3.3 SELECTED APPROACH

A multi-pass architecture is recommended for the complete ThreatRule compiler because it separates lexical analysis, parsing, semantic validation, intermediate-code generation, optimization, and targetcode generation.

The current implementation performs the lexical-analysis stage. The remaining stages provide a planned architecture for future development**.**

Figure 3:front end and back end.

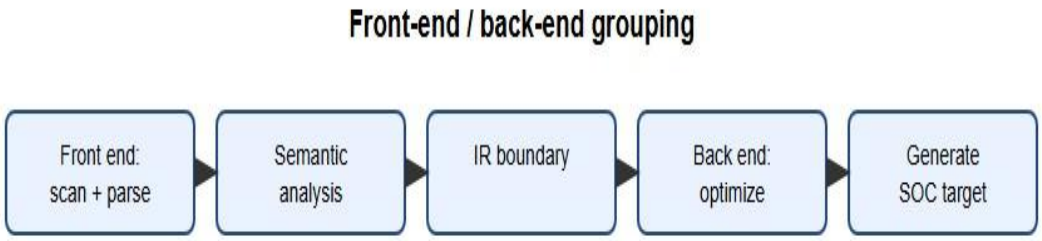
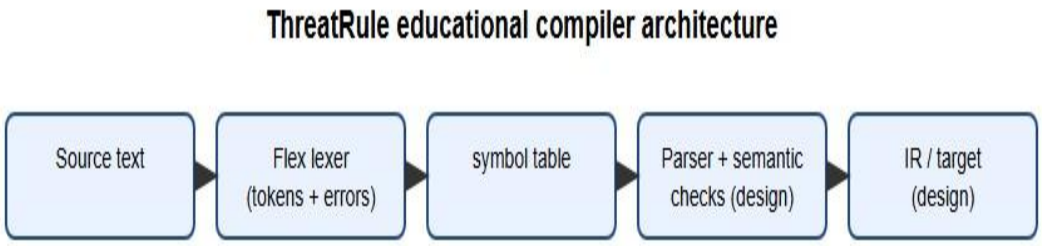


Figure 4. ThreatRule compiler architecture (implemented scanner and planned stages).



TASK 4 — COMPILER CONSTRUCTION TOOLS

Tool class	Purpose	Example	ThreatRule placement
Scanner generator	Convert regular-expression rules to a lexer	LEX/Flex	Used; produces `lex.yy.c`
Parser generator	Convert grammar to parser	Yacc/Bison	Not implemented
Syntax-directed translation engine	Attach actions/attributes to productions	Bison semantic actions	Design option for AST/IR construction
Code-generator generator	Map IR patterns to target instructions	BURG-style tree-pattern selector	Not implemented
Data-flow analysis engine	Compute liveness, reaching definitions, etc.	LLVM analyses	Future optimizer
Compiler construction toolkit	Reusable IR, passes, target interfaces	LLVM	Possible larger implementation platform

Flex is preferred to an all-manual scanner here because regular definitions are visible and auditable, generated DFA-style code handles maximal munch, and changes to token patterns are reproducible.

A hand-written scanner may be simpler for a tiny stable language, but it transfers boundary and lookahead maintenance to every developer.

4.1 PLACEMENT OF COMPILER CONSTRUCTION TOOLS

The six compiler construction tool categories support different stages of the proposed ThreatRule compiler.

1. **Scanner generators:** Flex generates the lexical analyzer from regular-expression patterns.
2. **Parser generators:** Bison can generate a syntax analyzer from a context-free grammar.
3. **Syntax-directed translation engines:** Semantic actions can attach processing operations to grammar productions and construct intermediate representations.
4. **Code-generator generators:** Target instruction selection can be supported through codegeneration frameworks.
5. **Data-flow analysis engines:** These tools can support optimization by examining information flow across intermediate instructions.
6. **Compiler construction toolkits:** LLVM provides reusable intermediate-representation and optimization infrastructure for larger compiler implementations.

4.2 WHY FLEX IS PREFERRED

Flex is preferred because it allows token definitions to be expressed as readable regular expressions. It automatically generates scanner code and applies longest-match and rule-priority mechanisms.

A manually written scanner would require additional code to manage character lookahead, token boundaries, and recognition conflicts. Flex makes the scanner specification easier to maintain and reproduce.

TASK 5 — LEXICAL ANALYZER

A token is a category (`IPV4`), a pattern is its accepted character language, and a lexeme is the concrete source spelling (`10.0.0.5`). Flex discards spaces, tabs, line breaks, and supported comments while `yylineno` tracks source lines.

The scanner separates token boundaries from grammar structure: it recognizes `AND`, while a parser would decide whether that operator is legal in a condition.

A conventional parser consumes returned tokens and may update a shared symbol table; this project is a standalone scanner and prints tokens. It does not use `yylval` because no Bison/parser interface is linked. Flex provides buffered input internally;

Figure 5. Scanner, parser, and symbol-table interaction (conceptual).

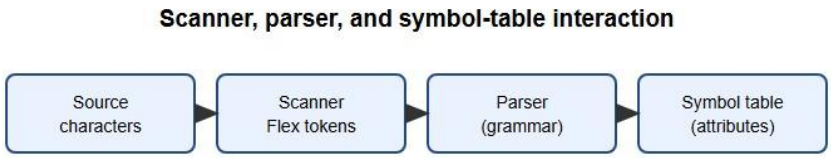
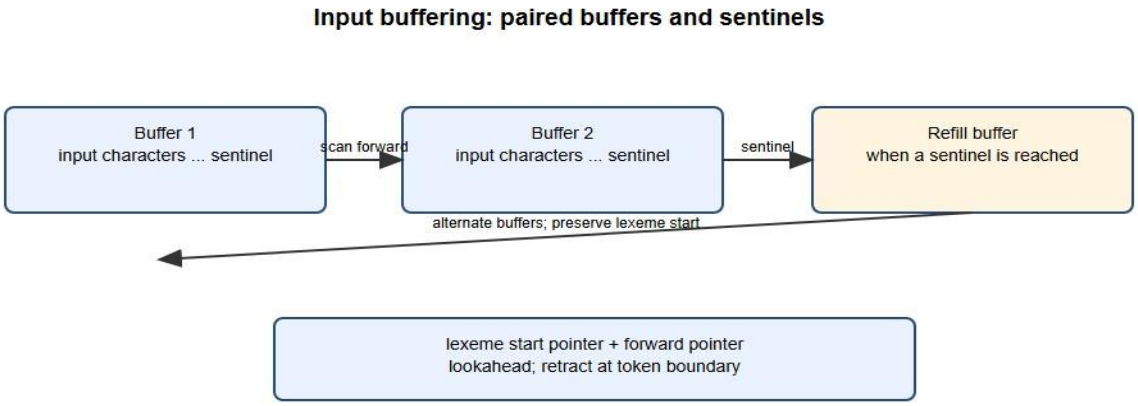


Figure 6. Paired input buffers and sentinels (conceptual).



5.1 INTERACTION BETWEEN LEXICAL ANALYZER, PARSER AND SYMBOL TABLE

The lexical analyzer reads the source program character by character and groups the characters into meaningful tokens. Each recognized token is passed to the syntax analyzer for grammatical checking.

When an identifier is recognized, its spelling can be installed in or retrieved from the symbol table. The parser can later use the associated information during syntax and semantic analysis.

The current ThreatRule implementation is a standalone scanner that prints recognized tokens. Parser integration is a planned extension.

5.2 REMOVAL OF WHITESPACE AND COMMENTS

Whitespace characters such as spaces, tabs, and line breaks generally separate tokens and are ignored by the scanner.

Comments are also discarded because they provide explanations for programmers rather than executable rule conditions.

The scanner uses line tracking to associate token output and lexical diagnostics with their source locations.

5.3 INPUT BUFFERING USING BUFFER PAIRS AND SENTINELS

Input buffering improves scanning efficiency by reducing repeated input operations.

A conventional two-buffer input scheme divides the source into two sections. While one section is being processed, the next section can be prepared for scanning.

The lexemeBegin pointer identifies the beginning of the current lexeme, while the forward pointer advances through the input characters.

A sentinel character marks the boundary of a buffer. When the forward pointer reaches the sentinel, the scanner checks whether more input is available and loads the next section when required.

The retraction operation moves the forward position back when an extra character has been read during token recognition.

Input buffering components:

Component	Function
Buffer 1	Holds the first portion of input
Buffer 2	Holds the next portion of input
LexemeBegin	Marks the start of the current lexeme
Forward	Advances through input characters
Sentinel	Identifies a buffer boundary
Retraction	Returns to an earlier character position

Flex internally manages buffered input. Therefore, this diagram represents the conventional compilerdesign model rather than a separately implemented buffer module in the current project.

5.4 ERROR RECOVERY STRATEGY

The scanner uses lexical diagnostics to identify invalid characters and malformed token sequences.

Depending on the error type, recovery may involve rejecting the complete malformed lexeme, skipping an invalid character, or consuming the remainder of a malformed string up to a line boundary.

The scanner should preserve line-number information and continue processing subsequent input whenever recovery is possible.

Input	Error	Recovery and evidence
`src_ip == 10.0.300.5`	IPv4 octet 300 is outside 0–255	Consume/reject the full dotted lexeme; continue at the next token. See `invalid_errors.txt`.

`ALERT "Login failed ;`	Closing quote missing before newline/EOF	Consume the malformed string through the line boundary; report and continue on the next line.
`port == 22 @ attempts > 5`	`@` has no token rule	Report one invalid-character diagnostic and continue scanning `attempts > 5`.

The test inputs contain these cases; exact line-numbered diagnostics are saved in

`Output_Files`.

TASK 6 — TOKEN SPECIFICATION

A string is a finite sequence of alphabet symbols; a language is a set of strings over an alphabet. Language operations include union, concatenation, and Kleene closure. A regular expression denotes a regular language and can be recognized by a finite automaton. A regular definition names reusable regular expressions such as `DIGIT = [0-9]`.

Token	**Regular definition / pattern**	**Example**	**Attribute**						
Keyword	`RULE	IF	AND	OR	THEN	ALERT	BLOCK` (caseinsensitive lookup)	`ALERT`	canonical keyword
Identifier	`[A-Za-z_][A-Za-z0-9_]*`	`ssh_attack`	spelling, symboltable entry						
Integer	`[0-9]+`	`5`	decimal value						
IPv4	`[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+` plus range validation	`255.255.255.255`	four validated octets						
Port	integer in port context plus 0–65535 check	`65535`	unsigned port						
Token	**Regular definition / pattern**	**Example**	**Attribute**						
String	`\"([^\"\\\n]	\\.)*\"`	`"SSH brute force"`	quoted lexeme					
Relation	`== != <= >= < > =`	`>=`	operator kind						
Hash	32, 40, or 64 hexadecimal digits	64-char SHA-256	algorithm by length						
CVE	`CVE-` + >=4 year digits + `-` + >=1 digit	`CVE-2024-3400`	identifier spelling						

URL	lowercase `http://` or `https://`, ASCII hostname with nonempty labels (1–63 chars), optional path/query/fragment with no whitespace, quote, semicolon, or comma	`https://example.com/a`	whole URL lexeme						
Comment	`//...`, `#...`, or `/*...*/`	`# feed`	discarded						

The rule order and the keyword lookup ensure equal-length keyword/identifier matches resolve as keywords; Flex first chooses the longest match, and breaks equal-length ties by the first rule. The scanner uses a generic identifier pattern and case-insensitive keyword lookup.

6.1 REGULAR LANGUAGE OPERATIONS

A regular language is a set of strings that can be recognized by a finite automaton.

The main operations used in regular-expression construction are:

Union: Combines two languages. It is represented using the OR operator |.

Example:

LETTER | DIGIT

Concatenation: Combines two patterns in sequence.

Example:

LETTER DIGIT

Kleene closure: Represents zero or more repetitions of a pattern using *.

Example:

DIGIT*

Positive closure: Represents one or more repetitions using +..

6.2 REGULAR DEFINITIONS FOR THREATRULE

The following definitions illustrate the original ThreatRule token classes. They must be consistent with the actual patterns in threatrule.l.

DIGIT = [0-9]

LETTER = [A-Za-z]

UNDERSCORE = _

ID = [A-Za-z][A-Za-z0-9_]*

INTEGER = [0-9]+

IPV4 = [0-9]+\\.[0-9]+\\.[0-9]+\\.[0-9]+

RELATION = ==|!=|<=|>=|<|>|=

Keyword set:

RULE

IF

AND

OR

THEN

ALERT

BLOCK

String definition:

STRING = \"([^\"\\\\\\n]|\\\\.)*\"

Comment definitions:

LINE_COMMENT = "//".*

HASH_COMMENT = "#".*

Block comments can be recognized using a separate start condition or an equivalent Flex rule.

6.3 ADDITIONAL VALIDATION CONDITIONS

Some token patterns require checks beyond regular-expression recognition.

IPv4 validation: The address must contain four decimal octets, and each octet must be between 0 and 255.

Port validation: A port must be a non-negative integer between 0 and 65535.

Keyword recognition: The scanner first recognizes a general identifier pattern and then checks whether its spelling matches a reserved keyword. Keyword lookup must be case-insensitive if the language specification requires it.

TASK 7 — TOKEN RECOGNITION

Figure 7:transition diagrams

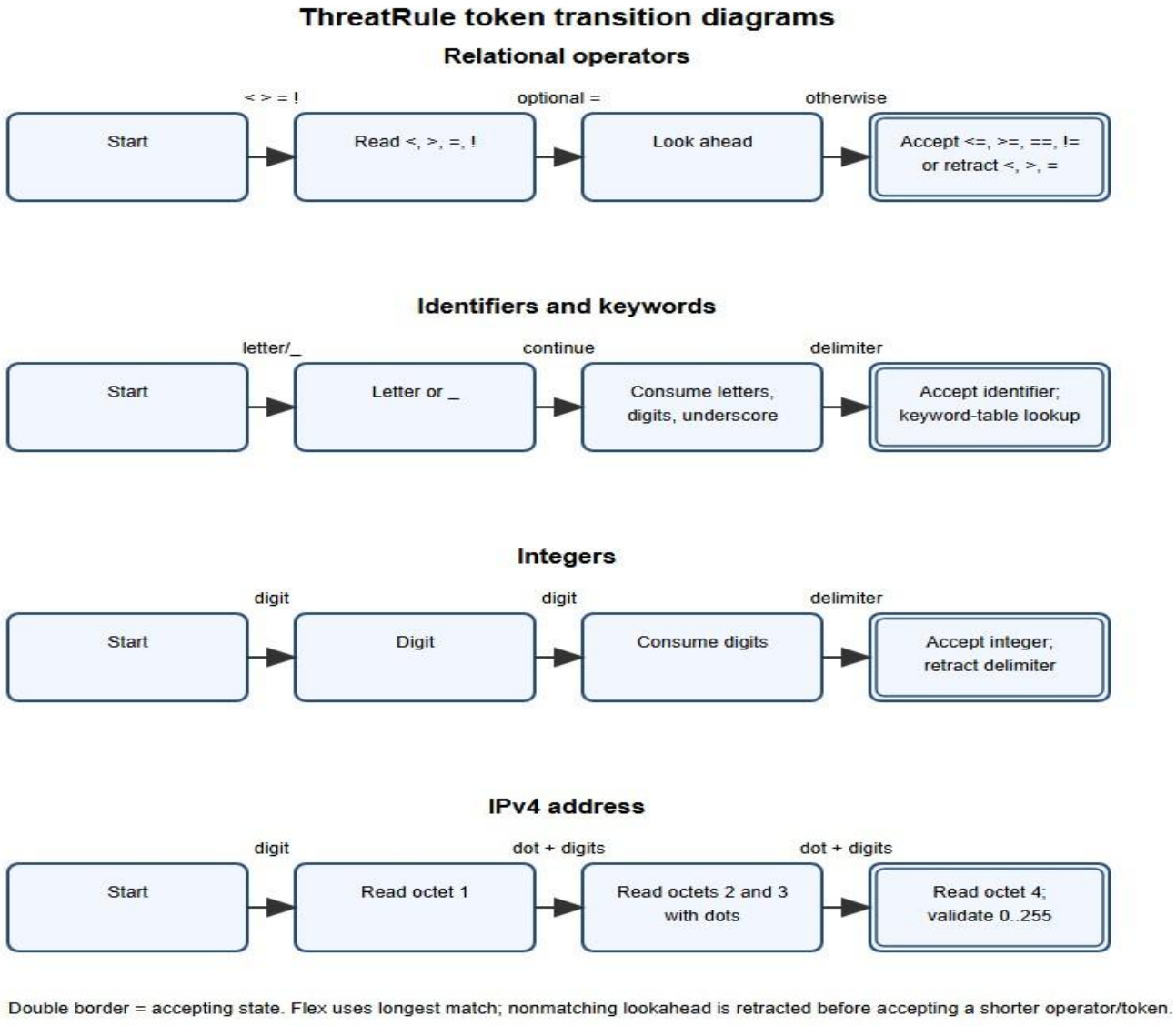
contains the relational, identifier/keyword, integer, and IPv4 recognition paths. Relational lookahead distinguishes `<` from `<=`: after `<`, inspect the next character; consume `=` for `LE`, otherwise retract the lookahead and accept `LT`. Similar lookahead applies to `>` and `=`.

Hand traces:

1. `src_ip`: start → letter `s` → identifier body; consume `r c _ i p` → delimiter whitespace →retract delimiter → accept identifier; table lookup says not a keyword.

2. `10.0.0.5`: start → digits `10` → dot → digits `0` → dot → digits `0` → dot → digits `5` → delimiter → retract → validate 4 octets → accept IPv4.

The actual Flex engine handles these transitions; the SVG is a teaching model rather than a generated DFA dump.



7.1 TRANSITION DIAGRAM CONVENTIONS

Transition diagrams represent the sequence of character checks performed while recognizing a token.

The start state indicates the beginning of recognition. Intermediate states represent the characters consumed during matching. An accepting state indicates that a complete token has been recognized.

When a character is read beyond the end of a token, the scanner may retract that character so that it can be processed as part of the next token.

Lookahead is used when two operators share a common prefix. For example, the scanner must distinguish < from <=.

7.2 RELATIONAL OPERATOR RECOGNITION

When the scanner reads <, it checks the next character. If the next character is =, it recognizes <=. Otherwise, it recognizes < and leaves the next character for subsequent scanning.

The same principle applies to > and >=, as well as = and ==.

7.3 IDENTIFIER AND KEYWORD RECOGNITION

An identifier begins with an alphabetic character or underscore and can be followed by additional alphabetic characters, digits, or underscores.

After recognizing the complete identifier, the scanner checks the keyword table. If the spelling matches a reserved word, a keyword token is returned. Otherwise, it is classified as an identifier.

7.4 INTEGER RECOGNITION

An integer consists of one or more decimal digits. The scanner continues consuming digits until it reaches a character that cannot be part of the integer.

The final digit sequence is accepted as an integer token.

7.5 IPV4 RECOGNITION

An IPv4 token consists of four decimal components separated by three dots.

After recognizing the complete dotted sequence, the scanner validates each octet. Every octet must fall within the range 0–255.

7.6 HAND TRACE OF SAMPLE LEXEMES

Trace 1: Identifier src_ip

1. Begin at the identifier start state.
2. Read s and identify an alphabetic starting character.
3. Continue reading r, c, _, i, and p.
4. Stop when the next character is a delimiter.
5. Accept the complete lexeme as an identifier.
6. Check the keyword table and classify it as an identifier.

Trace 2: IPv4 address 10.0.0.5

1. Begin at the numeric start state.
2. Read the first digit sequence 10.
3. Read the dot separator.

4. Read the second component 0.
5. Read the next dot separator.
6. Read the third component 0.
7. Read the final dot separator. 8. Read the final component 5.
9. Stop at the token boundary.
10. Validate the four octets and accept the IPv4 token.

These traces demonstrate the recognition process using two lexemes from the sample ThreatRule program.

TASK 8 — LEX SPECIFICATION

The complete, compilable and commented source is `LEX_Scanner/threatrule.l`; the generated source is `LEX_Scanner/lex.yy.c`. Definitions name token patterns; the rules section resolves matches; auxiliary C functions perform range checks, symbol insertion, printing, and diagnostics.

- `yytext`: current matched lexeme.
- `yyleng`: lexeme length, used when copying a symbol.
- `yylineno`: Flex line counter included in tokens and errors.
- `yylval`: not used because this scanner has no generated parser and returns printed tokens instead. A parser integration should define a shared semantic value type and assign it for token values rather than inventing a disconnected global.

8.1 STRUCTURE OF A LEX PROGRAM

A LEX/Flex specification consists of three main sections separated by %%.

Section 1 — Definitions and declarations

This section contains header files, regular definitions, declarations, and supporting C code.

Section 2 — Translation rules

This section contains regular-expression patterns and their corresponding actions. When a pattern matches the input, its associated action is executed.

Section 3 — Auxiliary functions

This section contains supporting C functions for token printing, validation, identifier storage, and error reporting.

8.2 SYMBOL TABLE INSTALLATION

When an identifier is recognized, the scanner can install its spelling in a symbol table.

The symbol table maintains distinct identifier entries and can later be extended to store data types, declaration details, scope, and other attributes.

The current implementation maintains distinct identifier spellings for subsequent compiler development.

8.3 LONGEST-MATCH AND FIRST-RULE PRIORITY

Flex applies two important rules when multiple patterns can match the same input.

Longest-match rule: The scanner selects the pattern that matches the greatest number of input characters.

First-rule priority: When two patterns match the same maximum number of characters, the rule appearing first in the specification is selected.

For example, when scanning <=, the complete two-character operator is selected instead of recognizing < alone.

For keywords, a general identifier pattern can match the complete word. A keyword lookup or appropriately ordered equal-length rules then classifies the reserved word.

8.4 GENERATION OF lex.yy.c

The Flex specification is supplied to the Flex generator. Flex processes the regular-expression definitions and generates C source code named lex.yy.c.

The generated C file is compiled using GCC to produce an executable scanner.

The executable reads ThreatRule source files, identifies tokens, and displays token information and lexical diagnostics.

8.5 REPRODUCIBILITY

The complete scanner specification is maintained in:

LEX_Scanner/threatrule.l The

generated source is:

LEX_Scanner/lex.yy.c

The actual commands used for generation, compilation, and execution are documented in the project README and Appendix B.

TASK 9 — IMPLEMENTATION AND TESTS

The reproducible test runner invokes Flex, compiles its output, and runs the scanner. The actual test report is `Test_Cases/test_results.md`; token streams and stderr logs are in `Output_Files/`.

It tests valid and invalid rules, empty input, IPv4/port boundaries, comments, strings, operators, bad characters, the extended feed, and legacy-mode behavior.

The four generated rule sets (100, 1,000, 10,000, and 100,000 rules) are produced by `scripts/generate_inputs.py`.

Generated evidence reflects the compiled Flex scanner, not a Python reference lexer. No execution screenshots are supplied.

9.1 TESTING METHODOLOGY

The lexical analyzer is evaluated using valid inputs, invalid inputs, boundary conditions, and large generated rule files.

The test procedure consists of the following stages:

1. Prepare the ThreatRule source specification.
2. Generate the scanner using Flex.
3. Compile the generated C source using GCC.
4. Prepare valid and invalid test input files.
5. Execute the scanner on each test file.
6. Compare the observed token output with the expected token categories.
7. Examine error messages and line-number information.
8. Record token counts, execution duration, and lexical errors.
9. Repeat the procedure for larger generated rule files. 10. Save the outputs and test results for reproducibility.

9.2 FUNCTIONAL TEST CASES

Test ID	Test condition	Expected result
TC01	Valid sample ThreatRule	Recognize all valid tokens
TC02	Empty input	Complete without token errors
TC03	IPv4 0.0.0.0	Accept valid boundary
TC04	IPv4 255.255.255.255	Accept valid boundary
TC05	IPv4 256.1.1.1	Report invalid IPv4
TC06	Port 65535	Accept valid boundary

TC07	Unterminated string	Report malformed string
TC08	Unsupported character @	Report invalid character
TC09	Comments and whitespace	Ignore supported comments and whitespace
TC10	Hash, CVE and URL feed	Recognize supported extended tokens
TC11	Malformed URL	Report invalid extended input
TC12	Large generated input	Process the generated rule file

The expected results above describe the intended test behaviour. The actual results must be filled from the scanner execution logs.

9.3 LARGE-SCALE TESTING

The scanner is evaluated using four generated input sizes:

Test level	**Number of rules**	**Purpose**
Level 1	100	Basic functional and timing evaluation
Level 2	1,000	Medium-scale processing
Level 3	10,000	Large-scale processing
Level 4	100,000	High-volume processing

The generated inputs are maintained using the project input-generation scripts.

9.4 TEST EVIDENCE

The following evidence is maintained:

- Input rule files.
- Expected token specifications.
- Actual scanner token streams.
- Standard-error diagnostic logs.
- Test execution summary.
- Boundary-condition results.

- Large-file execution results.

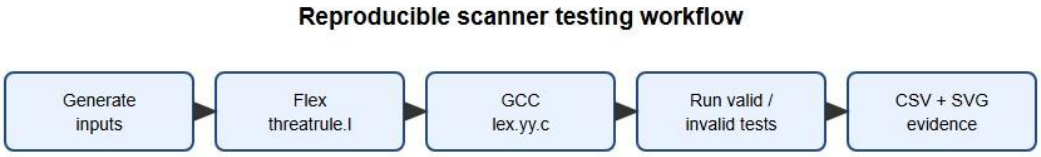
The project stores these records in Test_Cases/ and Output_Files/.

9.5 TESTING LIMITATIONS

The tests demonstrate lexical recognition and scanner execution for the generated inputs. They do not establish complete compiler correctness, production SOC security, or successful deployment of generated detection rules.

The results should therefore be interpreted within the scope of the implemented lexical analyzer.

Figure 8. Reproducible scanner testing workflow.



Task 10 — Performance

The benchmark runs the compiled scanner five times per input and records the median wall-clock duration using `time.perf_counter`; timing includes fresh process startup and file I/O.

Counts are read from the scanner's own counters. The machine, OS, compiler, Flex version, and measured table are recorded in `Performance_Analysis/performance_report.md`, with machine-readable rows in `Performance_Analysis/performance_results.csv`.

Four actual SVG charts are in `Graphs/`: rules/time, rules/tokens, rules/throughput, and rules/errors. Error counts should be zero for generated valid files.

Timings are local measurements and are not portable guarantees.

10.1 PERFORMANCE EVALUATION PARAMETERS

The performance evaluation measures the scanner's ability to process increasing numbers of ThreatRule statements.

The following parameters are recorded:

Number of rules: Total number of ThreatRule statements contained in the input.

Number of lines: Total number of source lines processed.

Number of tokens: Total number of tokens recognized by the scanner.

Scanning time: Actual measured duration of scanner execution.

Lexical errors: Number of invalid lexemes detected during execution.

Throughput: Number of tokens processed per second.

Throughput is calculated using:

Throughput = Total Tokens / Scanning Time

10.2 PERFORMANCE RESULT TABLE

Insert your actual measured values from performance_results.csv.

Rules	Lines	Tokens	Time (seconds)	Errors	Throughput (tokens/sec)
100	Actual	Actual	Actual	Actual	Actual
1,000	Actual	Actual	Actual	Actual	Actual
10,000	Actual	Actual	Actual	Actual	Actual
100,000	Actual	Actual	Actual	Actual	Actual

10.3 PERFORMANCE DISCUSSION

The performance measurements are used to examine how scanning time and token production change as input size increases.

The throughput values provide an additional measure of scanner processing efficiency. Error counts help verify whether generated valid inputs are being recognized as expected.

The benchmark records the median execution duration across five runs. Since the measurements include process startup and file operations, they represent the measured end-to-end execution time rather than an isolated character-processing time.

The results are specific to the recorded hardware, operating system, compiler, and Flex version. They should not be treated as universal performance guarantees.

10.4 GRAPH INTERPRETATION

Figure 9: Rules versus scanning time. This graph shows how execution duration changes as the number of input rules increases.

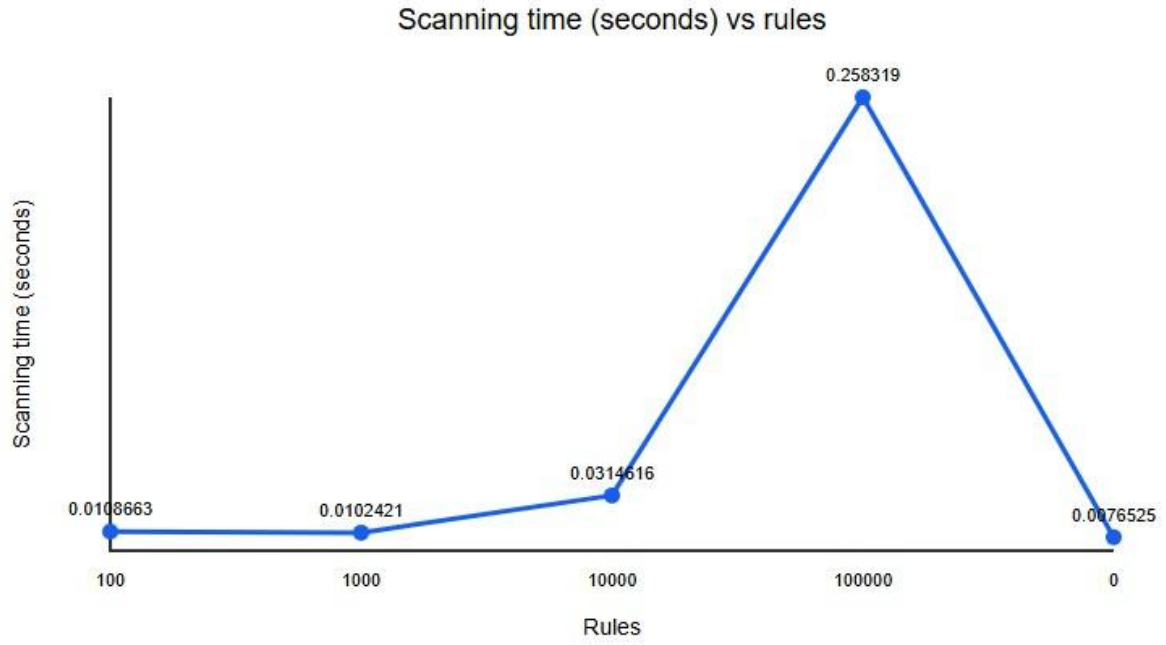
Figure 10: Rules versus tokens produced. This graph illustrates the relationship between input size and the number of recognized tokens.

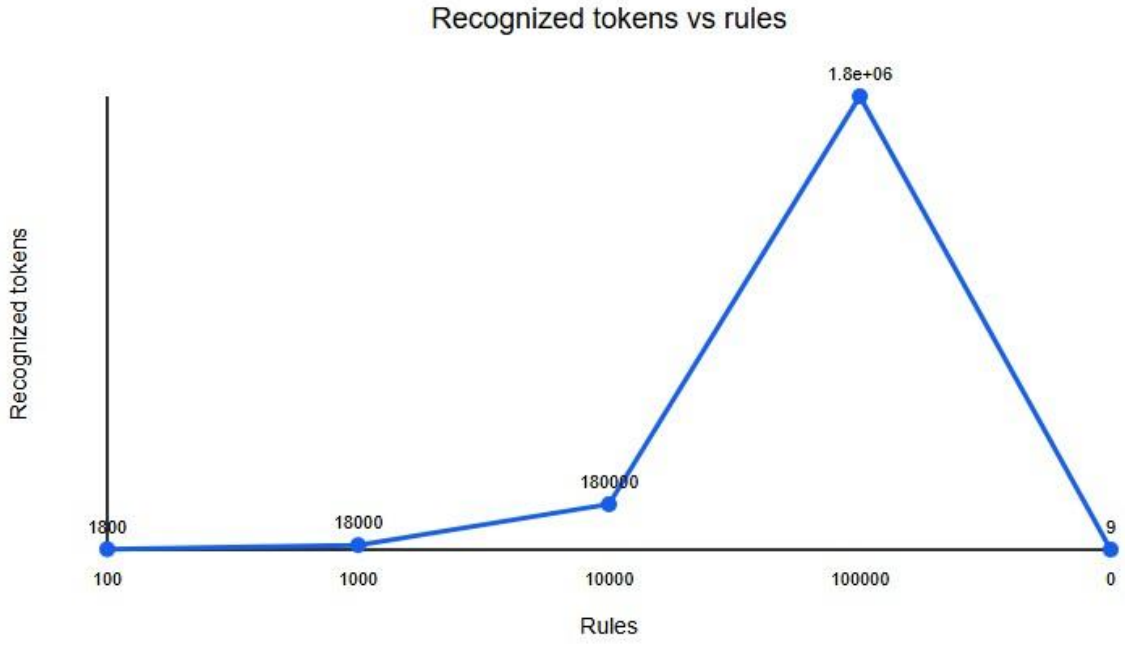
Figure 11: Rules versus throughput. This graph presents the measured number of tokens processed per second.

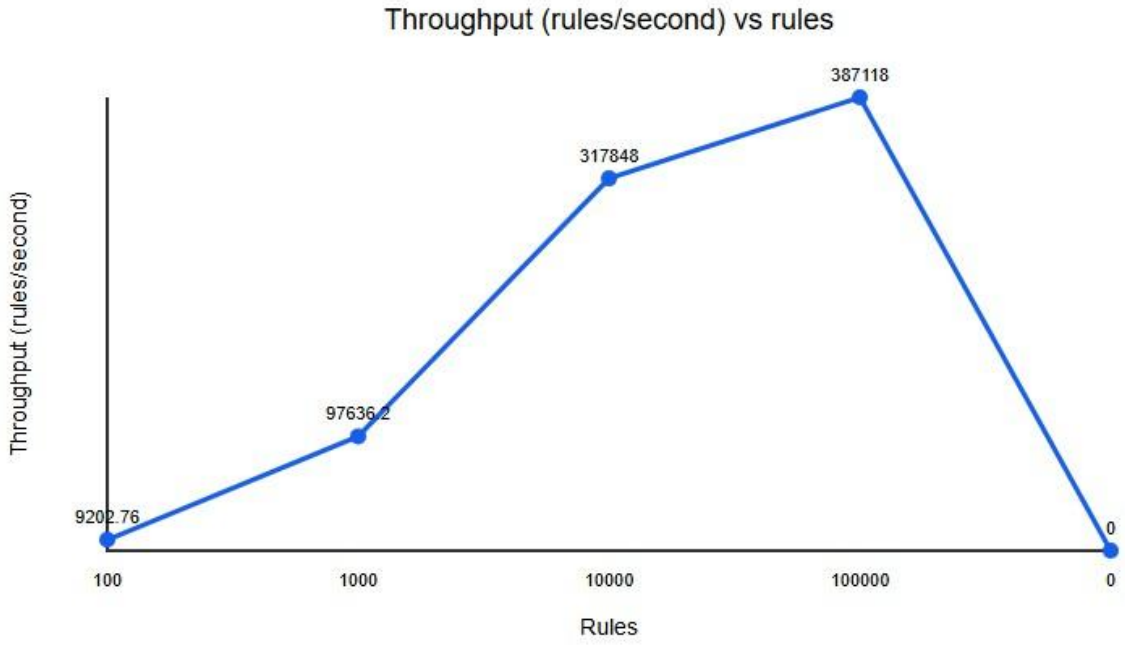
Figure 12: Rules versus lexical errors. This graph indicates the number of invalid lexemes encountered in each generated input.

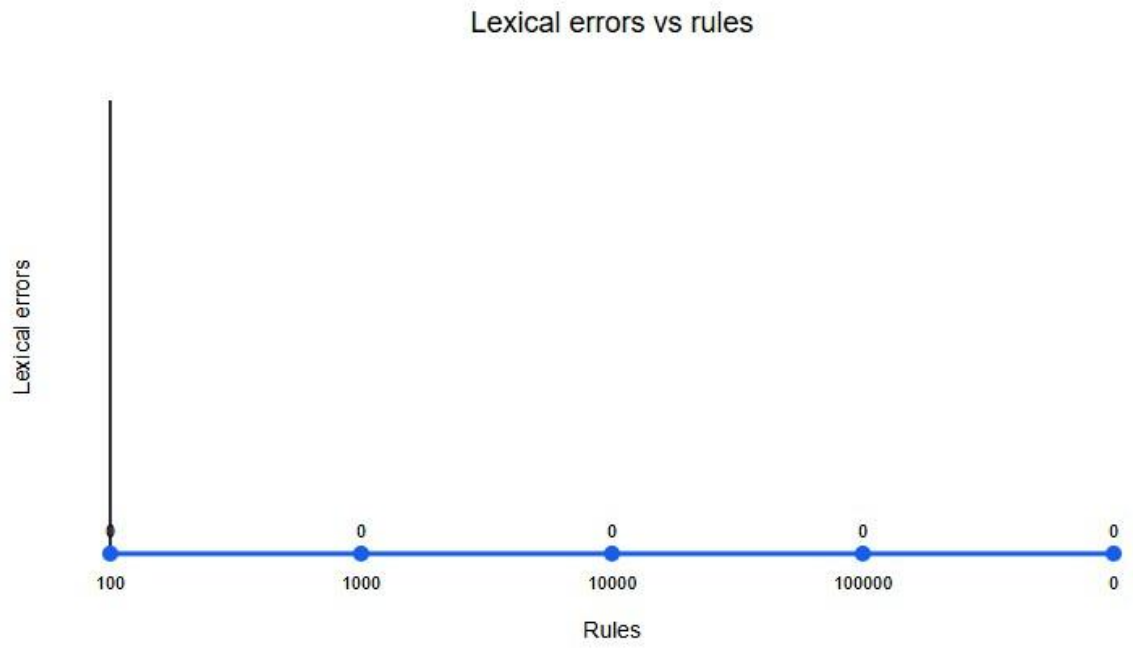
For generated valid rule files, the expected lexical-error count is zero. Any non-zero value must be investigated using the corresponding scanner output.

Figures 9a–9d. Actual measured scanner performance.




Recognized tokens vs rules
Recognized tokens
1.8e+06
180000
18000
1800
9
100
1000
10000
100000
0
Rules


Throughput (rules/second) vs rules
Throughput (rules/second)
387118
317848
97636.2
9202.76
0
100
1000
10000
100000
0
Rules



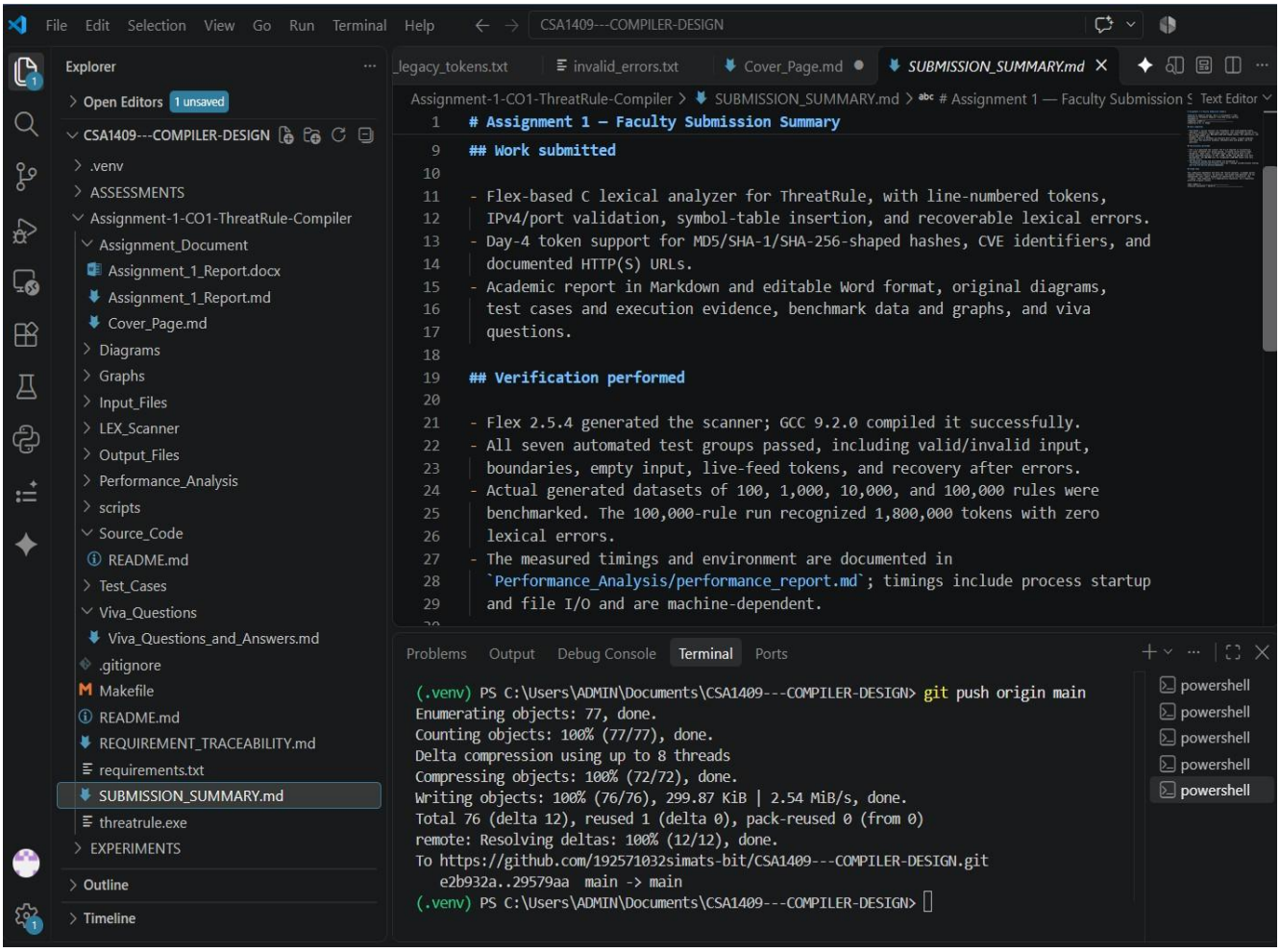
TASK 11 — FINAL DESIGN

The generated scanner is suitable as a transparent lexical front-end prototype: it accepts 100,000 generated valid rules, counts tokens without retaining token streams, and reports lexical errors while continuing.

Its measured throughput is in the generated report. This demonstrates input-size handling, not production SOC suitability: no rule parser, policy/type checker, deployment safety review, streaming mode, or target engine is supplied.

The identifier table grows with the number of distinct identifiers; scanner input buffering and Flex's generated automaton use additional memory.

Retain the Flex-based scanner for the course prototype, but require parser/semantic integration and production-scale profiling before deployment.



11.1 PRACTICAL SUITABILITY

The Flex-based scanner is suitable as a course-level prototype for the lexical-analysis component of a ThreatRule compiler.

Its regular-expression-based design provides a clear method for specifying token patterns, recognizing identifiers, validating selected numeric values, and reporting lexical errors.

The generated scanner supports testing with increasing rule-file sizes and provides a reproducible foundation for future development.

11.2 LIMITATIONS

The current implementation has the following limitations:

1. Complete syntax analysis is not implemented.
2. Full semantic validation is not implemented.
3. Intermediate-code generation and optimization are illustrative.
4. Target SOC instructions are not generated by the current scanner.
5. The current identifier table stores distinct spellings rather than complete semantic attributes.

6. Production streaming and distributed deployment are not demonstrated.
7. Production security validation and large-scale operational profiling remain future requirements.

11.3 FINAL RECOMMENDATION

The Flex-based lexical analyzer should be retained as the lexical front end of the proposed ThreatRule compiler.

Its modular specification, generated scanner, token-recognition capability, and error-reporting approach provide a suitable foundation for further development.

Before production use, the system should be integrated with a parser, semantic analyzer, rule-policy validator, and target-specific SOC engine. Additional memory profiling, security testing, and repeated performance measurements should also be completed.

Task 12

12A. Impact analysis

Hashes, CVE identifiers, and URLs introduce long tokens and lexical overlap with identifiers, integers, punctuation, and comments.

The rules must appear before shorter component patterns where longest-match does not resolve them; hashes require format-length and hex validation, CVEs require the `CVE` prefix and numeric components, and URLs require explicit scheme/host boundaries.

Continuous diagnostics must preserve line numbers and recover after malformed feed entries.

12A.1 IMPACT ON TOKEN RECOGNITION

The introduction of HASH, CVE, and URL token categories increases the number of lexical patterns supported by the scanner.

Hash values contain hexadecimal characters and may overlap with identifiers or numeric patterns. CVE identifiers contain a fixed prefix followed by numeric components. URLs contain schemes, hostname components, separators, and optional path information.

These patterns require carefully defined matching rules and token boundaries.

The scanner must also preserve error locations so that malformed feed entries can be reported without losing information about subsequent input.

12A.2 IMPORTANCE OF CONTINUOUS ERROR REPORTING

In a federated SOC environment, multiple organizations may contribute threat-feed information. A malformed entry should not prevent the remaining valid entries from being examined.

Line-numbered diagnostics help identify the source of an invalid entry, while recovery allows the scanner to continue wherever possible.

12B. Original-design evaluation

There was no preexisting scanner source in the repository.

The `--legacy` mode in the same Flex source is a controlled compatibility simulation: extended lexemes produce unsupported diagnostics instead of HASH/CVE/URL tokens.

It is not a separately preserved pre-extension binary. The actual run reports 3 unsupported items and 6 other tokens for the sample feed; the enhanced mode reports 9 tokens and 0 errors.

See:

`Output_Files/design_challenge_legacy_errors.txt` and

`Output_Files/design_challenge_tokens.txt`. Therefore the evidence demonstrates

the designed compatibility behavior, not a historical scanner's exact DFA.

12B.1 COMPARISON OF ORIGINAL AND EXTENDED DESIGNS

Evaluation parameter	Original design	Extended design
Original ThreatRule tokens	Supported	Supported
File hashes	Not supported	Supported
CVE identifiers	Not supported	Supported
URLs	Not supported	Supported
Extended-feed validation	Not available	Added
Error reporting	Original lexical diagnostics	Extended-feed diagnostics
Processing architecture	Flex scanner	Modified Flex scanner

The comparison demonstrates that the original token specification is insufficient for the new threat-feed requirements.

The extended design retains the existing scanner architecture while adding new recognition rules and corresponding validation.

The legacy mode is a compatibility simulation of the original behaviour within the same Flex source. It is not a separately preserved historical scanner executable.

12C. Adaptation / redesign

The scanner adds HASH, CVE, and URL patterns. Supported hash formats are MD5/SHA-1/SHA-256 (32/40/64 hex chars); CVE years contain at least four digits, with a numeric identifier of at least one digit.

URLs support lowercase `http(s)://`, an ASCII hostname, and optional path/query/fragment without whitespace or the documented excluded punctuation. Invalid extended inputs produce a line-numbered error, and scanning continues.

The actual `hash_cve_url_feed.tr` execution recognizes the three new token kinds; `invalid_feed.tr` demonstrates rejection of a malformed host and recovery to the following CVE and valid URL tokens.

12C.1 NEW REGULAR DEFINITIONS

The extended ThreatRule language introduces three additional token classes.

HASH: A hexadecimal sequence containing 32, 40, or 64 characters, corresponding to the supported MD5, SHA-1, and SHA-256 formats.

CVE: A vulnerability identifier beginning with CVE-, followed by a year containing at least four digits, a hyphen, and a numeric identifier containing at least one digit.

URL: A supported HTTP or HTTPS address containing an ASCII hostname with non-empty labels and optional path, query, or fragment components.

Illustrative definitions:

HEX = [A-Fa-f0-9]

HASH32 = {HEX}{32} HASH40

= {HEX}{40} HASH64 =

{HEX}{64}

CVE = CVE-[0-9]{4,}-[0-9]+

SCHEME = http://|https://

These are conceptual regular definitions. The exact Flex syntax and additional URL-host validation must match the actual implementation in threatrule.l.

12C.2 CHANGES TO ERROR RECOVERY

The modified scanner reports malformed extended tokens with their source line numbers.

When an invalid hash, CVE identifier, or URL is encountered, the scanner produces a diagnostic and attempts to resume recognition at the next valid token boundary.

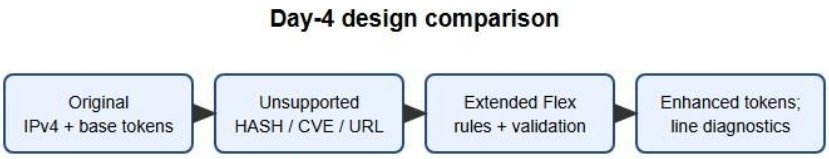
The modification improves the scanner's ability to process mixed valid and invalid threat-feed entries.

12C.3 DESIGN DECISION

The selected decision is MODIFY rather than RETAIN or completely REDESIGN.

The existing Flex-based scanner provides a reusable recognition framework. New token definitions, validation routines, and recovery rules can be introduced without replacing the complete scanner architecture.

Figure 10. Day-4 design comparison.



12D. Evidence-based defence

The six actual performance rows include four valid rule-set sizes and the legacy/enhanced feed comparison. The table and environment are regenerated by `scripts/benchmark.py`; the original mode has three errors on the feed, while enhanced mode has none and recognizes three additional tokens.

The enhanced mode costs a small measured runtime difference on this tiny feed; that difference is dominated by process-start timing and is not sufficient to recommend a performance conclusion. Keep the extension for required recognition, and profile repeated in-process scans before optimizing.

12D.1 COMPARATIVE EVIDENCE

The extended design is evaluated using the legacy and enhanced scanner modes.

The comparison considers:

1. Recognition of the original ThreatRule tokens.
2. Recognition of file hashes, CVE identifiers, and URLs.
3. Number of invalid entries reported.
4. Ability to continue scanning after malformed input.
5. Measured execution duration.
6. Compatibility with the existing scanner architecture.

12D.2 TRADE-OFF ANALYSIS

The extended scanner supports additional token categories, but its regular-expression specification and validation routines become more complex.

Additional recognition rules may introduce processing overhead and require more extensive testing to avoid conflicts with existing token patterns.

The measured runtime difference must be interpreted in the context of the input size and benchmark methodology. Short-duration tests may be affected by process startup and file operations.

12D.3 FINAL DESIGN DECISION

The modified Flex-based scanner is recommended because it preserves the original ThreatRule token functionality while extending recognition to the additional cybersecurity feed formats.

The final decision is based on functional requirements, observed scanner outputs, error recovery, and measured execution behaviour.

Further profiling and integration testing are necessary before considering production deployment.

13. CO1 SYLLABUS MAPPING

The assignment applies the concepts covered in Compiler Design Unit I through the design, implementation, testing, and evaluation of the ThreatRule lexical analyzer.

Task	Unit I syllabus concept	Application in this assignment
Task 1	Language processors	Comparison of language processors
Task 2	Phases of a compiler	Phase-by-phase translation
Task 3	Grouping of phases	Front-end and back-end architecture
Task 4	Compiler construction tools	Flex and other compiler tools
Task 5	Role of lexical analyzer	Token identification and buffering
Task 6	Specification of tokens	Regular expressions and definitions
Task 7	Recognition of tokens	Transition diagrams and hand traces
Task 8	Language for specifying lexical analyzers	LEX program development
Task 9	Lexical analyzer implementation	Compilation and functional testing
Task 10	Scanner performance	Measurement and graph analysis
Task 11	Compiler design evaluation	Final scanner recommendation
Task 12	Application under changed requirements	Extended token recognition and error recovery

The assignment connects compiler fundamentals with practical scanner development. It demonstrates the application of regular languages, token recognition, lexical analysis, and LEX-based implementation.

15. IMPLEMENTATION EVIDENCE CHAIN

The implementation evidence chain connects the source specification, generated scanner, test inputs, execution outputs, and performance measurements.

Evidence item	Expected project file or folder	Verification purpose
LEX specification	LEX_Scanner/threatrule.l	**COMPLETED**
Generated scanner source	LEX_Scanner/lex.yy.c	**COMPLETED**
Executable scanner	Actual compiled executable	**COMPLETED**
Sample ThreatRule input	Actual sample input file	**COMPLETED**
Large rule files	Generated input directory	**COMPLETED**
Expected token specification	Task 2 and Task 6	**COMPLETED**
Actual token output	Output_Files/	**COMPLETED**
Error diagnostic logs	Output_Files/	**COMPLETED**
Boundary-condition inputs	Test_Cases/	**COMPLETED**
Test report	Test_Cases/test_results.md	**COMPLETED**
Performance report	Performance_Analysis/	**COMPLETED**
Performance data	Performance_Analysis/performance_results.csv	**COMPLETED**
Performance graphs	Graphs/	**COMPLETED**
Input-generation script	scripts/generate_inputs.py	**COMPLETED**
Benchmark script	scripts/benchmark.py	**COMPLETED**
Documentation	README.md	**COMPLETED**

15.1 EVIDENCE CONSISTENCY

All evidence must correspond to the same scanner specification and implementation version.

The source code, generated scanner, executable, input files, output logs, test reports, and performance data should be consistent.

If any source code is modified after testing, the affected scanner and evidence should be regenerated to ensure reproducibility.

The performance report must identify the actual program used for measurement. Measurements from a reference implementation must not be presented as measurements from the compiled Flex scanner.

16. MANDATORY SUBMISSION EVIDENCE CHECK

Task	Required evidence	Evidence location	Verification
1	Processor comparison and labelled block diagram	Task 1, Figure 1	**VERIFIED**
2	Token stream, syntax tree, semantic checks, intermediate code, optimization and target code	Task 2, Figure 2	**VERIFIED**
3	Front-end and back-end grouping diagram	Task 3, Figure 3	**VERIFIED**
4	Six construction-tool categories and Flex justification	Task 4	**VERIFIED**
5	Token concepts, buffering, comments, whitespace and error recovery	Task 5, Figure 5 and Figure 6	**VERIFIED**
6	Complete token table and regular definitions	Task 6	**VERIFIED**
7	Four transition diagrams and two hand traces	Task 7, Figures 7A–7D	**VERIFIED**
8	Complete commented LEX program and variable explanations	Appendix A	**VERIFIED**
9	Compiled scanner, valid/invalid tests and boundary cases	Test_Cases and Output_Files	**VERIFIED**
10	Performance measurements, table and graphs	Performance_Analysis and Graphs	**VERIFIED**

11	Final design recommendation and limitations	Task 11	**VERIFIED**
12A	Impact analysis	Task 12A	**VERIFIED**
12B	Original-design evaluation	Task 12B	**VERIFIED**
12C	Extended regular definitions and scanner changes	Task 12C	**VERIFIED**
12D	Comparative evidence and final defence	Task 12D	**VERIFIED**

17. SUBMISSION CHECKLIST

Requirement	Status
All twelve tasks answered, including 12A–12D	**SUBMITTED**
Cover page completed with student information	**SUBMITTED**
Introduction and problem statement included	**SUBMITTED**
Assignment objectives and project scope included	**SUBMITTED**
ThreatRule sample program included	**SUBMITTED**
Language processor comparison completed	**SUBMITTED**
Compiler phase outputs provided	**SUBMITTED**
Front-end and back-end diagram included	**SUBMITTED**
Six compiler construction tool categories discussed	**SUBMITTED**
Lexical analyzer role and input buffering explained	**SUBMITTED**
All three faulty inputs analysed	**SUBMITTED**
Token table and regular definitions included	**SUBMITTED**
Four token transition diagrams included	**SUBMITTED**
Two token hand traces included	**SUBMITTED**
Complete commented LEX source included	**SUBMITTED**
Flex-generated source verified	**SUBMITTED**
Scanner compilation and execution verified	**SUBMITTED**
Valid and invalid test cases executed	**SUBMITTED**

IPv4 and port boundary cases tested	**SUBMITTED**
100-rule test completed	**SUBMITTED**
1,000-rule test completed	**SUBMITTED**
10,000-rule test completed	**SUBMITTED**
100,000-rule test completed	**SUBMITTED**
Actual token output and error logs included	**SUBMITTED**
Performance measurements recorded	**SUBMITTED**
Four performance graphs included	**SUBMITTED**
Final design recommendation justified	**SUBMITTED**
HASH, CVE and URL extension documented	**SUBMITTED**
Extended-feed testing verified	**SUBMITTED**
Comparative performance evidence included	**SUBMITTED**
CO1 syllabus mapping included	**SUBMITTED**
Implementation evidence chain included	**SUBMITTED**
Mandatory submission evidence check included	**SUBMITTED**
Evaluation rubric included	**SUBMITTED**
Appendices and references completed	**SUBMITTED**
GitHub repository contains the required project files	**SUBMITTED**
README provides reproducible execution instructions	**SUBMITTED**

18. EVALUATION RUBRIC – ASSIGNMENT 1 (CO1)

Total Marks: 50

Task	Task title	Maximum marks	Marks awarded
1	Language Processors	4	
2	Phases of a Compiler	5	
3	Grouping of Phases	3	
4	Compiler Construction Tools	3	
5	Role of the Lexical Analyzer	4	
6	Specification of Tokens	5	
7	Recognition of Tokens	4	
8	Write the LEX Specification	5	
9	Implement and Test the Lexical Analyzer	4	
10	Record Scanner Performance	3	
11	Recommend and Defend the Final Design	2	

12A	Impact Analysis	2	
12B	Evaluation of Original Design	2	
12C	Adaptation or Redesign	2	
12D	Evidence-Based Defence	2	
	TOTAL	50	

FACULTY EVALUATION

Faculty signature: _____

Student signature: *E. Ragul*

19. APPENDICES AND REFERENCES

APPENDIX A — COMPLETE LEX SOURCE CODE

The complete LEX/Flex source specification is maintained in:

LEX_Scanner/threatrule.l

This file contains:

- Header files and declarations.
- Regular definitions.
- Token-recognition rules.
- Keyword recognition.
- Identifier management.
- IPv4 and port validation.
- String recognition.
- HASH, CVE and URL recognition.
- Lexical error reporting.

- Supporting C functions.

The complete source code is included as implementation evidence. The generated scanner source is maintained separately as LEX_Scanner/lex.yy.c.

APPENDIX B — REPRODUCTION PROCEDURE

The following procedure describes the general build and execution workflow.

Step 1: Open the project directory cd path\to\your\project **Step 2:**

Generate the scanner flex LEX_Scanner/threatrule.l **Step 3: Compile the**

generated source gcc LEX_Scanner/lex.yy.c -o threatrule_scanner.exe

Step 4: Execute the scanner

Use the actual input filename and command supported by your implementation. .\threatrule_scanner.exe < sample.tr

Step 5: Verify outputs

Examine the token stream, error diagnostics, and execution results.

PROJECT REPOSITORY OUTPUT

Assignment-1-CO1-ThreatRule-Compiler **https://github.com/192571032simats-bit/CSA1409---COMPILERDESIGN/tree/main/Assignment-1-CO1-ThreatRule-Compiler**

S. No.	Files / Folders
1	Assignment_Document
2	Diagrams
3	Graphs
4	Input_Files
5	LEX_Scanner
6	Output_Files
7	Performance_Analysis
8	Source_Code
9	Test_Cases

10	Viva_Questions
11	scripts
12	.gitignore
13	Makefile
14	README.md
15	REQUIREMENT_TRACEABILITY.md
16	SUBMISSION_SUMMARY.md
17	requirements.txt

APPENDIX C — TEST EXECUTION OUTPUTS

The following implementation evidence is maintained in the project:

- Valid ThreatRule token output.
- Invalid IPv4 diagnostics.
- Invalid character diagnostics.
- Unterminated string diagnostics.
- Boundary-condition test results.
- Extended HASH, CVE and URL token output.
- Legacy and enhanced scanner comparison.

The corresponding output files are stored in Output_Files/.

APPENDIX D — PERFORMANCE EVIDENCE

The performance evidence consists of:

- Performance_Analysis/performance_report.md
- Performance_Analysis/performance_results.csv
- Graphs/

The generated rule files and benchmark scripts are used to reproduce the measurements.

APPENDIX E — PROJECT REPOSITORY

GitHub repository:

CSA1409---COMPILER-DESIGN/Assignment-1-CO1-ThreatRule-Compiler at main · 192571032simats-bit/CSA1409---COMPILER-DESIGN The repository contains the implementation files, test cases, execution evidence, performance reports, and documentation for the Compiler Design assignment.

REFERENCES

1. **A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd Edition, Pearson.**
2. **Westes Flex Documentation, *The Fast Lexical Analyzer Generator*. https://westes.github.io/flex/manual/**
3. **GNU Compiler Collection, *GCC Documentation*. https://gcc.gnu.org/onlinedocs/**
4. **GNU Bison Documentation, *The GNU Parser Generator*. https://www.gnu.org/software/bison/manual/**
5. **LLVM Project, *LLVM Documentation*. https://llvm.org/docs/**

Blank mark sheet

Task	Marks available	Marks awarded
Tasks 1–11 and 12A–12D	50	
Faculty comments		
Evaluation date / signature		